

DeepLabv3 논문리뷰

Semantic segmentation을 수행하는 Deep Convolutional Neural Networks(DCNNs)에 있어 다음과 같은 두가지 어려움이 존재한다.

1. 연속적인 pooling 작업 혹은 convolution strides 조절에 의한 정보 손실

두 작업은 굉장히 추상인 특성을 DCNNs에게 학습시키는데 사용할 수 있지만 세부적인 정보를 필요로 하는 dense prediction에 방해가 될 수 있다. 본 연구에서는 이 문제점을 해결하기 위해 atrous convolution을 사용했다. 확대된 convolution이라고도 불리는 atrous convolution은 기존 ImageNet으로 훈련된 모델의 마지막 몇개의 layer들을 제거하고 제거된 filter layer들을 upsampling하여 사용하는 방법을 일컫는다. 이는 기존 filter의 weight에 구멍을 뚫어 사용하는 것과 같다. 이를 통해 추가적인 매개변수들을 사용하지 않고 추론 결과의 해상도를 조절할 수 있다.

2. 제각각의 객체 크기

본 연구에서는 지금까지 이를 해결하기 위한 제시된 여러가지 방안 중 네가지를 살펴보도록 한다.

- **Image Pyramid**각기 다른 크기의 객체에 Image Pyramid를 사용하여 각기 다른 크기의 feature map을 얻어낸다.
- **Encoder-Decoder**Encoder에서 각기 다른 크기의 feature map을 얻어내고, 이것들의 해상도를 decoder에서 부분적으로 개선시킨다.
- **Extra modules**더 넓은 범위를 나타내는 정보를 얻기 위해 추가적인 layer를 쌓는다.
- **Spatial Pyramid Pooling**다양한 크기와 비율을 가진 pooling layer들을 적용해 다양한 scale의 객체들을 얻는다.

본 연구에서는 pooling 범위, 정보가 나타내는 넓이를 넓힐 수 있는 atrous convolution에 대해 cascaded modules과 spatial pyramid pooling에 대하여 적용한다. 특히 본 연구에서 사용한 atrous convolution은 다양한 비율과 batch normalization을 이용한다. 이러한 다양성은 학습에 있어 중요하다 생각한다. 또한, atrous convolution을 병행적 그리고 수직적, 두가지 방법으로 적용한다. 또한, 3×3 convolution에 대하여 높은 비율로 atrous convolution을 적용할 경우 발생하는 문제에 대해서도 다룬다.

Methods

해당 논문에서는 dense feature를 얻어내기 위해 atrous convolution을 어떻게 사용했는지에 대해 다루고, atrous convolution을 수직, 수평으로 배치한 모듈에 대해 다룬다.

Atrous Convolution for Dense Feature Extraction

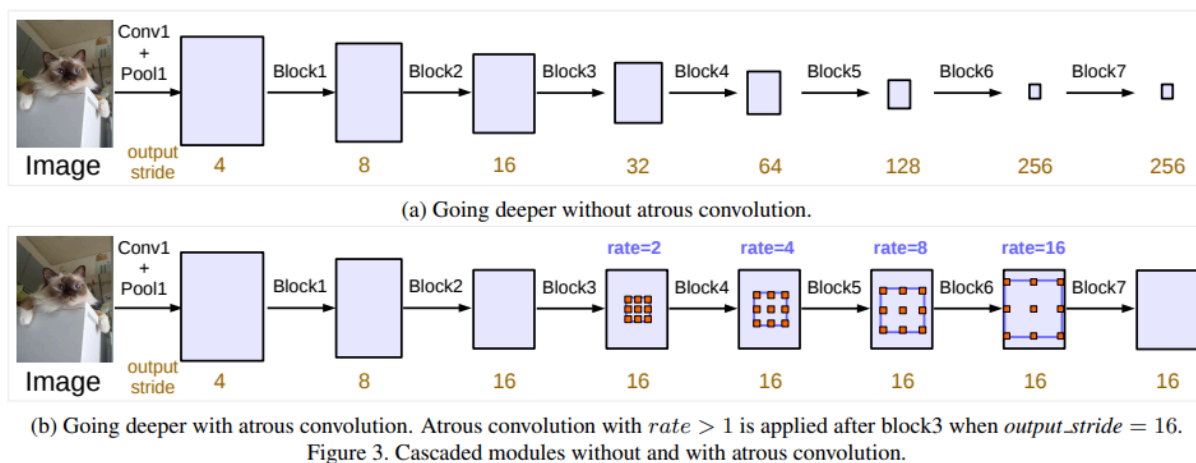
DCNNs는 fully convolutional한 방법을 사용해서 효율적으로 semantic segmentation을 수행한다. 그러나 반복되는 max-pooling 작업과 strind은 결과의 해상도를 낮춘다. Deconvolution 계층들이 낮아진 해상도를 높일 수 있지만 본 연구에서는 atrous convolution을 사용했다.

2차원의 입력 x , 출력 y , 필터 w 를 생각할 때, 위치 i 에 대하여 다음과 같은 연산이 수행된다. r 을 조절하며 filter의 시야(field-of-view)를 넓힐 수 있다.

$y[i] = \sum_k x[i+r \times k]w[k]$ (r 은 atrous rate임과 동시에 stride에 해당한다. 일반적인 convolution은 r 이 1이다.)

Atrous convolution을 이용하면 출력 이미지의 해상도를 직접적으로 조절할 수 있다. 여기서 output stride는 입력 이미지의 해상도와 출력 해상도의 비율을 의미한다. DCNNs의 입력 이미지와 fully-connected layer, global pooling을 적용하지 전 결과 이미지를 비교하면, 결과 이미지가 32배 작다는 것 즉, output stride가 32라는 것을 알 수 있다. 만약 출력의 공간적 밀집도(spatial density)를 2배로 높이고 싶다면, down sampling (decimation)을 피하기 위해 마지막 pooling 혹은 convoluiton의 stride를 1로 설정하고 하위의 모든 convolutional layer들을 r 이 2인 atrous convolution으로 대체하면 된다. 이러한 방식을 사용하면 추가적인 파라미터를 사용하지 않고도 출력의 밀집도를 높일 수 있다.

Going Deeper with Atrous Convolution



먼저 atrous convolution을 수직으로 적용한 부분을 살펴보겠다. ResNet의 마지막 모듈을 여러 개 복사한 후 DCNNs의 끝 부분에 수직으로 배치하였다. 해당 모듈들에는 각 3×3 convolution들이 포함되어 있고, 복사하여 덧붙인 모듈 중 마지막 모듈을 제외하고는 마지막 convolution의 stride는 2이다. 이와 같은 배열은 더 넓은 범위의 정보를 얻기 편하게 하는 데에 있다. 그러나 semantic segmentation에서는 이와 같은 작업이 세세한 정

보들을 삭제하기에 적합하지 않다. 따라서 atrous convolution을 사용해야 한다. 이때, 가장 바람직한 output stride는 16이다.

Atrous Spatial Pyramid Pooling (ASPP)

각기 다른 atrous rate를 가지고 ASPP를 진행하면 multi-scale한 정보를 효율적으로 얻어낼 수 있다. 그러나 rate이 커질수록 유효한 가중치들이 감소하는 것을 관찰할 수 있었다. 65×65의 입력에 3×3의 필터를 이용해서 atrous convolution을 진행하였을 때, atrous rate이 증가하여 입력크기와 비슷해 질수록, 예상하듯 더 넓은 범위의 정보를 얻어내는 것이 아니라 단순한 1×1필터를 이용한 효과와 비슷해지는 것을 확인할 수 있다. 이는 3×3 필터의 가중치 중 가운데 가중치만 의미있는 연산을 진행하기 때문이다. 이와 같은 문제를 극복하고 넓은 범위의 쓸모있는 정보를 얻어내기 위해서는 다음과 같은 절차를 거친다.

우선, global average pooling을 진행한다. pooling의 결과를 가지고 256의 필터로 이루어진 1×1 convolution을 진행한다. 이때, batch normalization도 진행한다. 이 결과값(result)을 바탕으로 마지막으로 하나의 1×1 convolution과 각각 6, 12, 18의 atrous rate를 가진(output stride가 16일 때) convolution을 병렬로 진행한다. 이후 각 branch들의 결과와 (result)를 concat한 후 1×1 convolution을 진행하여 마지막 logit을 산출해낸다.

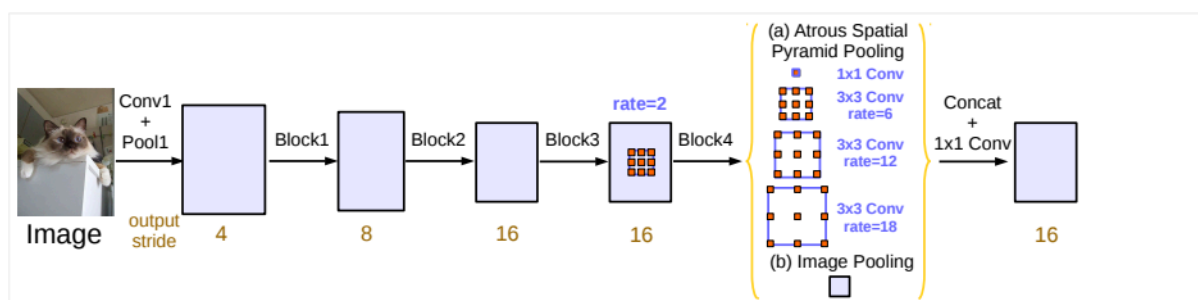


Figure 5. Parallel modules with atrous convolution (ASPP), augmented with image-level features.